

Trabajo Práctico 3: RISC-V Pipeline

Agustín Trachta

Agustín Pallardó

2 de abril de 2026

Índice

1. Introducción	2
2. Introducción	2
2.1. Objetivos del Trabajo Práctico	2
2.2. Descripción General del Sistema	2
3. Arquitectura del Procesador RISC-V	3
3.1. Formato de las Instrucciones	3
3.1.1. Ejemplo de decodificación	3
3.2. Estructura del Pipeline	4
3.3. Unidad de Control y Decodificación	4
3.4. Unidad de Detección de Riesgos (Hazards) y Forwarding	4
3.4.1. Valores y Lógica de Forwarding	4
3.4.2. Detección y Resolución de Hazards (Stalls y Flushes)	5
3.5. Ejemplos Analíticos de Riesgos de Datos en Ensamblador	5
3.6. Módulo de Comunicación UART y FIFOs	6
4. Debug Unit	6
4.1. Protocolo de Comunicación	6
4.2. Modos de Operación	6
4.2.1. Modo Continuo (CMD_RUN)	6
4.2.2. Modo Paso a Paso (CMD_STEP)	7
4.3. Limpieza, Vaciado del Pipeline y Reinicio	7
5. Resultados de Simulación	8
5.1. Casos de Prueba	8
5.2. Comportamiento ante la Ausencia de Instrucción HALT	8
5.3. Capturas de Simulación	9
6. Síntesis y Análisis de Temporización (Timing)	9
6.1. Reportes de Vivado	9
6.2. Identificación del Camino Crítico y Skew	10
6.3. Determinación de la Frecuencia Óptima	10
7. Conclusiones	11

1. Introducción

2. Introducción

2.1. Objetivos del Trabajo Práctico

El objetivo de este trabajo práctico es diseñar, implementar y validar un procesador *RISC-V* segmentado de cinco etapas sobre FPGA, incorporando además una interfaz de depuración y control externo mediante comunicación UART.

En primer lugar, se pretende construir un *pipeline* funcional capaz de ejecutar un subconjunto de instrucciones de la arquitectura RISC-V, asegurando el correcto transporte de datos y señales de control entre las distintas etapas. Esto implica no solo el diseño del camino de datos y la unidad de control, sino también la resolución de conflictos asociados a dependencias de datos, riesgos de control y sincronización entre etapas.

En segundo lugar, el trabajo busca dotar al sistema de mecanismos de observabilidad y depuración que permitan inspeccionar su estado interno durante la ejecución. Para ello, se incorpora una *Debug Unit* capaz de interactuar con el procesador mediante comandos enviados por UART, posibilitando tareas como la carga de programas en memoria, la ejecución paso a paso, la detención controlada, y el volcado de registros, memoria y latches del pipeline.

Finalmente, se busca validar el funcionamiento integral del sistema tanto en simulación como en FPGA, verificando que la ejecución de programas, el manejo de instrucciones especiales como *HALT*, y la interacción con la unidad de depuración se comporten de acuerdo con lo esperado. De esta forma, el trabajo no solo apunta a obtener un diseño funcional, sino también a comprender en profundidad la dinámica interna de un procesador segmentado y las herramientas necesarias para analizarlo y depurarlo.

2.2. Descripción General del Sistema

El sistema desarrollado consiste en un procesador basado en la arquitectura RISC-V de 32 bits, implementado como un *pipeline* de cinco etapas clásicas: *Instruction Fetch* (IF), *Instruction Decode* (ID), *Execute* (EX), *Memory Access* (MEM) y *Write Back* (WB). Cada etapa cumple una función específica dentro del ciclo de ejecución de instrucciones y se encuentra separada por registros intermedios que almacenan tanto datos como señales de control.

El procesador cuenta con memorias de instrucciones y datos, un banco de registros, lógica de forwarding y detección de hazards, y soporte para instrucciones aritméticas, lógicas, de acceso a memoria y de control de flujo dentro del subconjunto implementado. Además, incorpora señales de validez en los registros del pipeline, lo que permite distinguir entre instrucciones reales y burbujas introducidas por stalls o flushes, facilitando así un control más preciso de la ejecución.

Para permitir la interacción externa con el sistema, se utiliza una interfaz UART como canal de comunicación serie entre la FPGA y una computadora. A través de este enlace es posible enviar comandos y recibir respuestas desde un software de apoyo o una interfaz de usuario, sin necesidad de modificar directamente el hardware en cada prueba.

Sobre esta interfaz se apoya la *Debug Unit*, módulo encargado de coordinar la programación de la memoria de instrucciones, el control de ejecución del procesador y la recolección de información interna del sistema. Esta unidad interpreta los comandos recibidos por UART y genera las señales necesarias para iniciar o detener la ejecución, avanzar un ciclo, reiniciar el procesador o extraer información de depuración. Gracias a esta integración, el sistema no solo ejecuta programas, sino que también puede ser observado y analizado de manera detallada, lo cual resulta especialmente útil durante las etapas de prueba, validación y demostración del funcionamiento del procesador.

3. Arquitectura del Procesador RISC-V

3.1. Formato de las Instrucciones

La arquitectura RV32I utiliza instrucciones de 32 bits de longitud fija. Según el tipo de instrucción, esos 32 bits se organizan en distintos campos, como `opcode`, `rd`, `rs1`, `rs2`, `funct3`, `funct7` e inmediatos. Estos campos permiten identificar qué operación debe realizar el procesador, qué registros participan y si la instrucción requiere un valor inmediato.

Los formatos principales implementados en este procesador son: Tipo-R, Tipo-I, Tipo-S, Tipo-B, Tipo-U y Tipo-J.

Tipo	Campos
R	<code>funct7</code> <code>rs2</code> <code>rs1</code> <code>funct3</code> <code>rd</code> <code>opcode</code>
I	<code>imm[11:0]</code> <code>rs1</code> <code>funct3</code> <code>rd</code> <code>opcode</code>
S	<code>imm[11:5]</code> <code>rs2</code> <code>rs1</code> <code>funct3</code> <code>imm[4:0]</code> <code>opcode</code>
B	<code>imm[12 10:5]</code> <code>rs2</code> <code>rs1</code> <code>funct3</code> <code>imm[4:1 11]</code> <code>opcode</code>
U	<code>imm[31:12]</code> <code>rd</code> <code>opcode</code>
J	<code>imm[20 10:1 11 19:12]</code> <code>rd</code> <code>opcode</code>

Cuadro 1: Formatos principales de instrucciones en RV32I.

Cada formato responde a una necesidad distinta dentro del conjunto de instrucciones:

- **Tipo-R:** se utiliza para operaciones entre registros, como `add`, `sub`, `and`, `or`, `sll` o `slt`. En este caso, los campos `funct3` y `funct7` permiten distinguir la operación exacta.
- **Tipo-I:** se utiliza para instrucciones con inmediato, como `addi`, `andi`, `ori`, cargas desde memoria (`lb`, `lh`, `lw`, etc.) y también `jalr`. Incluye un inmediato de 12 bits.
- **Tipo-S:** se utiliza para escrituras en memoria, como `sb`, `sh` y `sw`. En este formato, el inmediato está dividido en dos partes dentro de la instrucción.
- **Tipo-B:** se utiliza para saltos condicionales, como `beq` y `bne`. El inmediato también aparece dividido, y luego debe reconstruirse para calcular la dirección de salto.
- **Tipo-U:** se utiliza en instrucciones como `lui`, donde se carga un inmediato de 20 bits en la parte superior del registro destino.
- **Tipo-J:** se utiliza en instrucciones de salto como `jal`, donde el inmediato permite calcular una dirección de destino más amplia.

Esta organización fija de los bits simplifica la etapa de decodificación, ya que permite extraer de manera ordenada los campos relevantes de cada instrucción y generar, a partir de ellos, tanto las señales de control como los operandos necesarios para la ejecución.

3.1.1. Ejemplo de decodificación

Por ejemplo, la instrucción `add x5, x1, x2` corresponde al formato Tipo-R. En este caso:

- `opcode` indica que se trata de una operación aritmética entre registros,
- `rd = x5` señala el registro destino,

- `rs1 = x1` y `rs2 = x2` indican los registros fuente,
- `funct3` y `funct7` determinan que la operación exacta es una suma.

De forma similar, en instrucciones como `addi x5, x1, 10`, el formato Tipo-I conserva los campos `rd` y `rs1`, pero reemplaza `rs2` y `funct7` por un inmediato de 12 bits, que luego será extendido y utilizado por la ALU.

3.2. Estructura del Pipeline

El núcleo del procesador implementa una arquitectura segmentada de 5 etapas: IF, ID, EX, MEM y WB. Estas etapas están separadas por registros intermedios o *latches* (`if_id_reg`, `id_ex_reg`, `ex_mem_reg`, `mem_wb_reg`).

En el archivo `top.v` se realiza la interconexión de todas ellas. Una característica importante de esta arquitectura es que la resolución de saltos incondicionales (JAL, JALR) y condicionales (BRANCH) se realiza de forma temprana en la etapa de decodificación (ID). Esto puede verse en las señales de redirección generadas en ID (`redirect_target_id`), que alimentan directamente al multiplexor del PC en la etapa IF. Gracias a esto, se reduce la penalización cuando un salto es tomado, en comparación con otros diseños que resuelven estas instrucciones recién en la etapa EX.

3.3. Unidad de Control y Decodificación

El recorrido de los datos y el comportamiento del procesador dependen principalmente de la Unidad de Control. Su implementación se concentra en dos submódulos instanciados dentro de la etapa de decodificación (`id_stage.v`):

- `control_unit.v`: A partir del `opcode` de 7 bits, genera las señales de control de la ruta de datos. Por ejemplo, al detectar el `opcode` `7'b0000011`, correspondiente a una instrucción de carga (LW), activa `MemRead = 1`, indica que el dato que se escribirá en el banco de registros proviene de memoria (`MemToReg = 1`) y selecciona el inmediato como segundo operando de la ALU (`ALUSrc = 1`) para calcular la dirección efectiva.
- `rv_decoder.v`: Se encarga de la decodificación fina de las operaciones de la ALU. Interpreta los campos `funct3` y `funct7` de las instrucciones Tipo-R y Tipo-I para determinar qué operación exacta debe realizarse, como suma, resta, desplazamientos o comparaciones.

3.4. Unidad de Detección de Riesgos (Hazards) y Forwarding

La ejecución en pipeline introduce dependencias entre instrucciones que deben resolverse para mantener el funcionamiento correcto del procesador. Para ello, el diseño incluye dos mecanismos principales: *forwarding*, que adelanta resultados entre etapas, y *stalling*, que detiene temporalmente el avance del pipeline cuando no es posible resolver un conflicto de otra manera.

3.4.1. Valores y Lógica de Forwarding

El módulo `forwarding_unit.v` evita ciclos de espera innecesarios adelantando resultados desde etapas posteriores (EX/MEM o MEM/WB) hacia las entradas de la ALU en la etapa EX. Para esto, genera dos señales de selección: `fwdA` para el operando `rs1` y `fwdB` para el operando `rs2`.

En la siguiente tabla se resumen los valores utilizados:

Valor (fwdA/B)	Tipo de Forwarding	Acción del Mux
2'b00	Sin forwarding	Se usa directamente el valor proveniente del banco de registros almacenado en el latch ID/EX.
2'b10	Desde EX/MEM	Se toma el resultado adelantado desde EX/MEM. Esto ocurre cuando la instrucción inmediatamente anterior ya calculó el dato necesario.
2'b01	Desde MEM/WB	Se toma el valor adelantado desde MEM/WB. Se usa cuando no aplica forwarding desde EX/MEM.

Cuadro 2: Valores codificados para el MUX de forwarding de los operandos de la ALU.

3.4.2. Detección y Resolución de Hazards (Stalls y Flushes)

Cuando un conflicto no puede resolverse mediante forwarding, entra en acción el módulo `hazard_unit.v`.

En estos casos, la unidad detiene temporalmente el avance del pipeline activando `stall_if` y `stall_id`, lo que congela el Program Counter y la etapa de decodificación. Al mismo tiempo, puede inyectar una burbuja o *NOP* mediante `flush_idex = 1`. Las situaciones principales se resumen a continuación:

Clasificación	Dependencia	Solución aplicada
<i>Load-Use</i>	La instrucción en EX es un LW y su registro destino (<code>rd</code>) es utilizado inmediatamente por la instrucción en ID.	Se inserta 1 burbuja (+1 stall). Luego, el dato puede recuperarse mediante forwarding desde MEM/WB.
<i>Branch-EX</i>	Un Branch en ID necesita comparar un valor que todavía se está calculando en la etapa EX.	Se inserta 1 burbuja (+1 stall) para esperar a que el dato esté disponible.
<i>Branch-MEM</i>	Un Branch en ID depende de un dato cargado desde memoria y aún no disponible.	Se requieren 2 ciclos de espera .

Cuadro 3: Casos principales que generan stalls o flushes en el pipeline.

3.5. Ejemplos Analíticos de Riesgos de Datos en Ensamblador

Para mostrar de forma práctica cómo actúan estos mecanismos, se presentan algunos ejemplos en ensamblador RV32I.

Ejemplo 1: Forwarding continuo sin penalización

1. `add x1, x2, x3` ; Suma y guarda el resultado en x1
2. `add x4, x1, x5` ; Usa inmediatamente el valor de x1
3. `add x6, x1, x7` ; Vuelve a usar x1 en la instrucción siguiente

Sin forwarding, estas instrucciones generarían conflictos. Sin embargo, al ejecutar la línea 2, el valor de `x1` todavía no fue escrito en el banco de registros, pero la unidad de forwarding detecta que está disponible en EX/MEM, por lo que activa `fwdA = 2'b10`. Luego, en la línea 3, ese mismo valor ya se encuentra más adelante en el pipeline, y se utiliza forwarding desde MEM/WB con `fwdA = 2'b01`. De esta forma, no es necesario detener la ejecución.

Ejemplo 2: Load-Use Hazard

1. `lw x1, 0(x2)` ; Carga un dato desde memoria en x1
2. `add x3, x1, x4` ; Usa inmediatamente el valor cargado en x1

En este caso, la instrucción `lw` recién obtiene el dato al final de la etapa `MEM`, por lo que la instrucción siguiente no puede usarlo de inmediato. Como no es posible resolver este caso con forwarding en el mismo ciclo, la unidad de hazards activa `stall_if = 1`, `stall_id = 1` y `flush_idx = 1`, insertando una burbuja en el pipeline. Luego de ese ciclo de espera, el valor ya puede ser adelantado mediante forwarding desde `MEM/WB`.

3.6. Módulo de Comunicación UART y FIFOs

La comunicación con el sistema externo se realiza mediante una interfaz UART asincrónica, compuesta por tres módulos principales: `baud_gen.v`, `uart_rx.v` y `uart_tx.v`.

Como el reloj interno del sistema trabaja a frecuencias mucho mayores (85 a 100 MHz) que la velocidad de transmisión serie, fue necesario incorporar buffers intermedios mediante `fifo.v`.

Estas FIFOs se implementan como memorias circulares con dos punteros: uno de lectura (`r_ptr_reg`) y otro de escritura (`w_ptr_reg`). Esto permite almacenar temporalmente los datos y desacoplar la velocidad de la CPU de la velocidad de la UART. Así, aunque el host envíe comandos en ráfaga, estos quedan correctamente almacenados y se procesan en orden, evitando pérdidas o sobreescrituras mientras la CPU continúa con otras tareas.

4. Debug Unit

La unidad de depuración (*Debug Unit*) funciona como la máquina de estados principal encargada de controlar la ejecución de la CPU desde el exterior. Su objetivo es permitir la carga de programas, la ejecución paso a paso, la detención del procesador y la lectura de registros, memoria y latches internos, todo a través de la interfaz UART.

De esta manera, actúa como un puente entre el usuario y la CPU, facilitando las pruebas y el análisis del sistema sin depender de herramientas externas más complejas.

4.1. Protocolo de Comunicación

La comunicación se realiza mediante comandos y respuestas codificados en bytes hexadecimales. Los comandos son enviados desde el host hacia la FPGA, y las respuestas son enviadas desde la FPGA hacia el host como confirmación o devolución de datos.

4.2. Modos de Operación

La Debug Unit controla distintos modos de funcionamiento de la CPU a través de su máquina de estados y de la señal `pipeline_en_r`.

4.2.1. Modo Continuo (CMD_RUN)

Cuando el host envía el comando `0x20`, la Debug Unit pasa al estado `ST_RUN_CONTINUOUS` y habilita la ejecución continua del pipeline mediante `pipeline_en_r = 1`.

Si durante la ejecución se detecta una instrucción `HALT`, el sistema deja de incorporar nuevas instrucciones y pasa al estado `ST_DRAIN`. En este estado, el pipeline sigue avanzando únicamente para permitir que las instrucciones que ya estaban dentro terminen su recorrido de forma segura. Una vez que el pipeline queda vacío, se envía la respuesta `RESP_OK_RUN_END` al host.

Comando (Mnemónico)	Código Hex	Función
CMD_NONE	0x00	Comando nulo, sin efecto.
CMD_PROG_BEGIN	0x10	Indica el comienzo de la carga de un programa en memoria de instrucciones. Luego se envían los datos correspondientes.
CMD_PROG_END	0x11	Marca el final de la programación.
CMD_RUN	0x20	Inicia la ejecución continua del programa cargado.
CMD_STEP	0x21	Avanza la ejecución un paso de reloj.
CMD_STOP	0x22	Detiene la ejecución actual.
CMD_DUMP_REGS	0x30	Solicita el contenido completo del banco de registros.
CMD_DUMP_LATCHES	0x31	Solicita el estado interno de los registros del pipeline.
CMD_DUMP_MEM	0x32	Solicita el contenido de la memoria de datos.
CMD_CLEAR_IMEM	0x40	Limpia la memoria de instrucciones escribiendo instrucciones NOP.

Cuadro 4: Comandos de entrada del protocolo UART.

Respuesta (Acuse)	Código Hex	Significado
RESP_OK_CLEAR	0xC0	La memoria de instrucciones fue limpiada correctamente.
RESP_OK_PROG	0xC1	La carga del programa se realizó correctamente.
RESP_OK_STEP	0xC2	Se ejecutó correctamente un paso de reloj.
RESP_OK_STOP	0xC3	La CPU fue detenida correctamente.
RESP_OK_RUN_END	0xC4	El programa terminó su ejecución al detectar una instrucción HALT y vaciar el pipeline.
RESP_DUMP_*	0xD0-D2	Indican el inicio o desarrollo de una trama de volcado de datos.
RESP_ERR	0xEE	Indica que se recibió un comando inválido o inesperado.

Cuadro 5: Respuestas enviadas por la Debug Unit al host.

4.2.2. Modo Paso a Paso (CMD_STEP)

Cuando se envía el comando 0x21, la Debug Unit habilita el pipeline solo durante un ciclo de reloj, permitiendo observar el avance exacto de una instrucción o de una transición interna.

Este modo es especialmente útil para depuración, ya que permite analizar el comportamiento del procesador con mucho detalle, ciclo por ciclo.

4.3. Limpieza, Vaciado del Pipeline y Reinicio

Antes de cargar o ejecutar un programa, es importante limpiar correctamente tanto la memoria de instrucciones como los registros internos del pipeline para evitar comportamientos erróneos.

- **Limpieza de Memoria de Instrucciones (ST_PROG_CLEAR):** Se activa mediante el comando 0x40. La Debug Unit recorre toda la memoria de instrucciones utilizando un índice interno (`clear_idx`) y escribe en cada posición la instrucción 32'h00000013, que

corresponde a un NOP. Esto asegura que no queden instrucciones residuales de ejecuciones anteriores.

- **Reinicio del Pipeline (ST_RESET_EXEC):** Complementa el proceso de inicialización limpiando los latches y estados internos del pipeline mediante la señal `reset_exec_r = 1`. Su función es evitar que queden datos viejos en los registros intermedios y garantizar que cada nueva ejecución comience desde un estado limpio.

5. Resultados de Simulación

5.1. Casos de Prueba

Para validar el funcionamiento del procesador se desarrollaron distintos casos de prueba en simulación, tanto a nivel de módulos individuales como del sistema completo. En una primera etapa, se verificó el comportamiento de bloques fundamentales como la ALU, la memoria de datos, la memoria de instrucciones, la unidad de forwarding y la unidad de detección de hazards. Luego, se realizaron pruebas integradas sobre el pipeline completo utilizando programas en ensamblador RV32I especialmente diseñados para ejercitar los distintos caminos de ejecución.

Los casos considerados incluyeron operaciones aritméticas y lógicas entre registros, instrucciones con inmediato, accesos a memoria mediante `load` y `store`, y secuencias con dependencias de datos entre instrucciones consecutivas. También se probaron escenarios con riesgos del tipo *load-use*, dependencias en instrucciones de salto condicional y situaciones donde el forwarding resultaba suficiente para evitar detenciones innecesarias del pipeline.

Además, se verificó el funcionamiento de la *Debug Unit* mediante testbenches que simularon el intercambio de comandos por UART. Esto permitió comprobar la correcta recepción de órdenes como carga de programa, ejecución continua, ejecución paso a paso, detención, limpieza de memoria y volcado del estado interno del procesador. Estas pruebas fueron especialmente útiles para detectar errores de integración entre módulos que no aparecían al analizar cada bloque por separado.

Un aspecto importante del proceso de simulación fue la necesidad de depurar problemas reales de temporización lógica y de sincronización funcional. Entre ellos se destacaron la correcta propagación de datos a través del pipeline, la validación del vaciado completo ante una instrucción `HALT`, y la correspondencia entre la latencia esperada de la memoria y el momento en que los datos debían estar disponibles para escritura o forwarding.

5.2. Comportamiento ante la Ausencia de Instrucción `HALT`

Las simulaciones permitieron comprobar que el procesador mantiene un comportamiento estable aun cuando el programa cargado no contiene una instrucción `HALT`. En este caso, la *Debug Unit* nunca detecta la condición de finalización del programa, por lo que la CPU continúa ejecutando instrucciones de manera indefinida mientras permanezca habilitada en modo continuo.

Sin embargo, este comportamiento no produce fallos internos ni estados inválidos. Esto se debe a que, antes de programar una nueva secuencia de instrucciones, la memoria de instrucciones es limpiada mediante el estado `ST_PROG_CLEAR`, reescribiendo todas sus posiciones con instrucciones NOP. Como resultado, si el contador de programa avanza más allá del código útil cargado por el usuario, el procesador continúa leyendo instrucciones neutras y el pipeline sigue operando de forma predecible.

En estas condiciones, el *Program Counter* continúa incrementándose hasta recorrer toda la memoria direccionable y eventualmente reiniciar su recorrido por desborde natural, sin provocar

excepciones ni comportamientos erráticos dentro de la arquitectura implementada. Desde el punto de vista funcional, esto significa que la ejecución no termina por sí sola, pero el sistema sigue siendo controlable desde el exterior.

Este punto fue relevante durante la etapa de pruebas, ya que permitió confirmar que la finalización del programa no debía depender únicamente del agotamiento del código cargado, sino de una condición explícita de parada. También ayudó a validar la necesidad de contar con mecanismos externos de control, como `CMD_STOP`, para abortar la ejecución en cualquier momento desde la interfaz de depuración.

A su vez, el análisis de este caso permitió ajustar el comportamiento del sistema ante la detección de `HALT`. En particular, fue importante distinguir entre detener inmediatamente el ingreso de nuevas instrucciones y permitir que las que ya estaban dentro del pipeline terminaran su recorrido. Esta dificultad llevó a refinar la lógica de vaciado dinámico del pipeline, evitando soluciones rígidas basadas únicamente en una cantidad fija de ciclos.

5.3. Capturas de Simulación

A continuación, se referencian los *waveforms* obtenidos en la herramienta de simulación, los cuales permiten observar de manera directa el comportamiento temporal del procesador y de la unidad de depuración ante los distintos casos ensayados.

En estas capturas resulta especialmente útil mostrar:

- la evolución del *Program Counter*,
- el contenido de las instrucciones en cada etapa del pipeline,
- las señales de `stall`, `flush` y `forwarding`,
- la escritura en el banco de registros,
- los accesos a memoria de datos,
- y la transición de estados de la *Debug Unit*.

Estas observaciones permiten relacionar el comportamiento esperado a nivel teórico con su implementación efectiva en hardware descriptivo, y fueron fundamentales para verificar que la resolución de riesgos, saltos y detenciones estuviera ocurriendo en el momento correcto.

6. Síntesis y Análisis de Temporización (Timing)

6.1. Reportes de Vivado

Una vez validado el diseño a nivel funcional mediante simulación, se procedió a su síntesis e implementación en Vivado con el objetivo de evaluar el costo en recursos y su comportamiento temporal sobre FPGA. Los reportes generados por la herramienta permitieron analizar la utilización de LUTs, registros, memoria embebida y rutas críticas, así como también verificar si el sistema cumplía con las restricciones de reloj planteadas.

Este análisis no solo permitió comprobar que el diseño era sintetizable, sino también detectar puntos sensibles en la implementación. En particular, fue importante estudiar cómo impactaban sobre el timing la interconexión entre etapas del pipeline, la lógica de forwarding, la unidad de hazards y el acceso a memorias. También se revisaron advertencias metodológicas y de temporización que surgieron durante el flujo de implementación, ya que varias de ellas ayudaron a identificar zonas del diseño que requerían una revisión más cuidadosa.

Otro aspecto que debió considerarse fue la correspondencia entre la memoria inferida o instanciada y la latencia asumida por el pipeline. Durante el desarrollo, una de las dificultades más importantes estuvo asociada justamente a la lectura de memoria de datos, ya que una discrepancia entre la latencia esperada y la configurada en el bloque de memoria podía desplazar los datos un ciclo completo y producir resultados incorrectos en la etapa de escritura.

6.2. Identificación del Camino Crítico y Skew

En una arquitectura segmentada, el objetivo principal del pipeline es aumentar el rendimiento permitiendo que varias instrucciones avancen en paralelo. Sin embargo, el período mínimo de reloj sigue estando condicionado por el camino combinacional más largo del diseño, es decir, por el llamado *camino crítico*.

En este trabajo, el camino crítico se encontró asociado a la interconexión entre registros del pipeline y lógica de procesamiento involucrada en la ejecución de instrucciones, particularmente en los recorridos que incluyen cálculo de resultados, selección de datos mediante multiplexores y escritura posterior en registros. Según la configuración sintetizada, estos caminos pueden involucrar la salida de memoria, los registros intermedios del pipeline, la lógica de selección del dato de escritura y su retorno hacia el banco de registros.

Este resultado es coherente con la experiencia de desarrollo, ya que una parte importante de la depuración se centró precisamente en comprender cómo se propagaban los datos entre etapas y en qué ciclo exacto quedaban disponibles. La interacción entre memorias, registros de pipeline y lógica de selección resultó ser un punto especialmente sensible, tanto desde lo funcional como desde lo temporal.

Junto con el camino crítico, también resulta relevante considerar el *clock skew*, es decir, la diferencia de tiempo con la que el flanco de reloj llega a distintos registros del circuito. Aunque este efecto forma parte normal de cualquier implementación física, su influencia se vuelve más importante a medida que se exige una frecuencia de operación mayor.

Si el margen temporal disponible es demasiado reducido, el skew puede contribuir a violaciones de *setup time*, impidiendo que una señal llegue y se estabilice correctamente antes del siguiente flanco de reloj. Por eso, no alcanza con que la lógica sea funcionalmente correcta: también debe cumplir con las restricciones temporales impuestas por la tecnología y el ruteo físico sobre FPGA.

6.3. Determinación de la Frecuencia Óptima

A partir de los resultados de síntesis e implementación, se buscó determinar una frecuencia de operación que ofreciera un equilibrio entre rendimiento y robustez temporal. Para ello, se analizaron los valores de *slack* obtenidos en los reportes de timing, procurando mantener un margen positivo que garantizara un funcionamiento confiable del procesador.

Durante el desarrollo, se trabajó principalmente con relojes del orden de decenas de MHz, y se evaluó también el comportamiento del sistema frente a distintas configuraciones de frecuencia. Este análisis resultó importante no solo para validar la CPU en sí, sino también para asegurar la correcta interacción con módulos auxiliares como la UART y las memorias embebidas.

En este contexto, se observó que no siempre la máxima frecuencia teórica resultaba ser la opción más conveniente para la validación completa del sistema. En varias etapas del proyecto fue preferible priorizar una configuración estable y repetible, que permitiera verificar correctamente la ejecución del pipeline, el intercambio UART y la depuración interna, antes que exigir el límite absoluto de operación.

Por lo tanto, la frecuencia de trabajo adoptada se definió considerando no solo el cumplimiento de timing, sino también la estabilidad global del sistema, la reproducibilidad de las

pruebas y la correcta integración entre todos los módulos del procesador.

7. Conclusiones

La implementación del procesador RISC-V sobre FPGA permitió comprobar en la práctica los principales conceptos estudiados en arquitectura de computadoras, particularmente aquellos vinculados con segmentación de instrucciones, resolución de riesgos y control de ejecución. El desarrollo de una CPU RV32I segmentada en 5 etapas no solo implicó diseñar la ruta de datos y las señales de control, sino también enfrentar problemas reales de integración, temporización y depuración que no siempre resultan evidentes en un análisis puramente teórico.

Uno de los aportes más importantes del trabajo fue la incorporación de una *Debug Unit* capaz de interactuar con la CPU mediante UART, permitiendo cargar programas, ejecutar en modo continuo o paso a paso, detener la ejecución y obtener el estado interno del sistema. Esta herramienta resultó fundamental para observar el comportamiento real del procesador, validar hipótesis durante la depuración y detectar errores que hubieran sido mucho más difíciles de aislar sin una interfaz de inspección dedicada.

Asimismo, la implementación de mecanismos de *forwarding*, *stall* y *flush* permitió resolver correctamente los principales riesgos de datos y de control presentes en una arquitectura segmentada. Las pruebas realizadas mostraron que el procesador responde adecuadamente ante dependencias entre instrucciones, conflictos del tipo *load-use* y situaciones de salto condicional o incondicional, manteniendo la correctitud de la ejecución.

A lo largo del desarrollo también surgieron dificultades concretas que enriquecieron el aprendizaje del proyecto. Entre ellas pueden mencionarse la necesidad de ajustar la latencia efectiva de las memorias para que coincidiera con la esperada por el pipeline, la depuración del comportamiento del HALT para lograr un vaciado real y seguro del pipeline, y la validación del protocolo de comunicación UART en presencia de transferencias extensas como los volcados de registros o memoria. Resolver estos puntos fue clave para lograr una implementación robusta y coherente.

En conjunto, los resultados obtenidos muestran que el sistema desarrollado cumple con los objetivos propuestos y constituye una implementación sólida y funcional del procesador requerido. Más allá del cumplimiento del trabajo práctico, el proyecto permitió integrar conocimientos de diseño digital, arquitectura de computadoras, verificación funcional, temporización y comunicación entre hardware y software, ofreciendo una experiencia completa y muy cercana a un desarrollo real sobre FPGA.